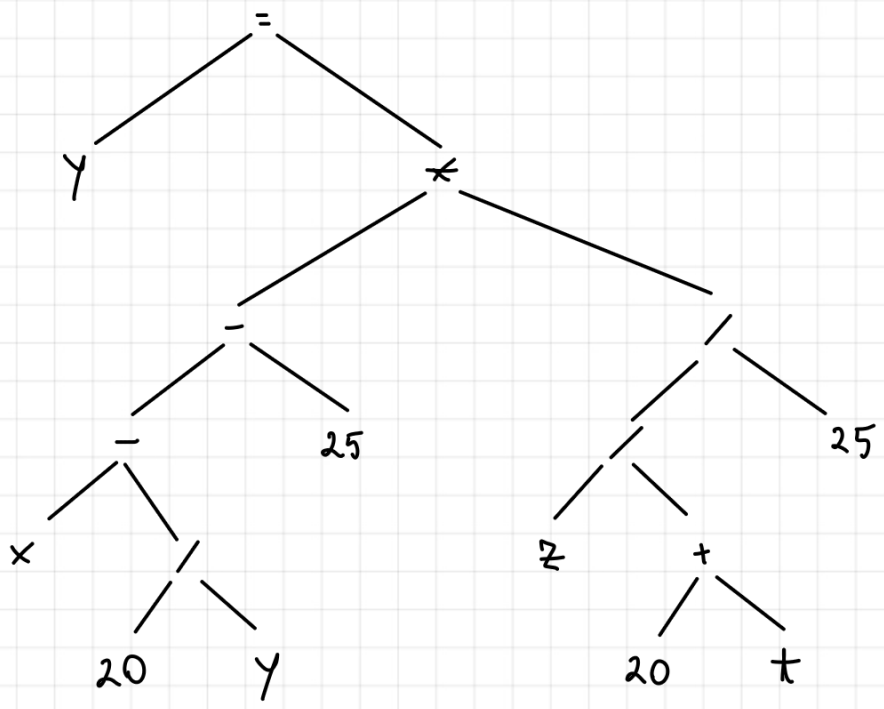
① $y = (x - (20/y) - 25) * (z / (20 + t) / 25)$

i)



ii)

x	id1
y	id2
z	id3
t	id4

temp1 = inttoreal(20)
temp2 = temp1 / id2
temp3 = id1 - temp2
temp4 = inttoreal(25)
temp5 = temp3 - temp4
temp6 = temp1 + id4
temp7 = id3 / temp6
temp8 = temp7 / temp4
temp9 = temp5 * temp8
id2 = temp9

iii)
temp1 = 20.0 / id2
temp2 = id1 - temp1
temp3 = temp2 - 25.0
temp4 = 20.0 + id4
temp5 = id3 / temp4
temp6 = temp5 / 25.0
id2 = temp3 * temp6

iv) (id 2) ⟶ R1 y

#20.0 / (R1) ⟶ R1 20/y

(id 1) ⟶ R2 x

(R2) - (R1) ⟶ R1 x - (20/y)

(R1) - #25.0 ⟶ R1 (x - (20/y) - 25)

(id 4) ⟶ R2 t

#20.0 + (R2) ⟶ R2 20+t

(id 3) ⟶ R3 z

(R3) / (R2) ⟶ R2 z/(20+t)

(R2) / #25.0 ⟶ R2

(R1) * (R2) ⟶ R1

(R1) ⟶ id2

(2.)

2. Consideră următorul cod:

```
int tab[3];
void p2(int *t, int &i)
{
   int aux;
   aux=t[i]; t[i]=i; i=aux;
}
void p1(int *t, int &i)
{
   int aux;
   p2(t,i);
   aux=i; i=t[i]; t[i]=aux;
   p2(t,i);
}
void main()
{
   i=0;
   tab[0]=3; tab[1]=1; tab[2]=2;
   p1(tab,i);
}
```

Care sunt valorile și cum au fost obținute pentru elementele tabloului tab după apelul procedurilor, dacă transmiterea parametrilor se face prin referință (&)?

tab

0	1	2
3	1	2

$i = 0$

p1(tab, 0)

→ tab[0] adr

=> p2(t , i) -> p2(3, 0)

aux = t[0] = 3

t[0] = 0

i = 3

=> tab

0	1	2
0	1	2

$i = 3$

aux = 3 , i = t[3] (nu există) -> Index Out of Bounds

(3.)

3. Se dă gramatica:
E-> E or T | T
T-> T and F | F
F-> (E) | true | false
Să se scrie acțiunile semantice care să calculeze valoarea expresiei logice E.

E → E or T { E.val = E1.val or T.val }
E → T { E.val = T.val }

T → T and F { T.val = T1.val and F.val }
T → F { T.val = F.val }

F → E { F.val = E.val }
F → true { F.val = true }
F → false { F.val = false }

④

4. Scrie semantica operațională pentru următorul cod:
`a=0; while(a<2025){b=0; while(b<2025) {a++;b++;}}`.
Imaginează un set de instrucțiuni de nivel scăzut și etichete pentru a scrie codul echivalent. Explică fiecare dintre instrucțiunile pe care le-ai utilizat.

```
a = 0;
while( a < 2025) {
    b = 0;
    while( b < 2025){
        a++;
        b++;
    }
}
```

while B do L end while

⇐

```
LOOP:
    IF (NOT B) GOTO OUT
    [CORP]
    GOTO LOOP
OUT:
```

```
a := 0
L1:
    if a >= 2025 goto EMB
    b := 0
L2:
    if b >= 2025 goto L1
    a := a + 1
    b := b + 1
    goto L2
EMB: HALT
```

```
    MOV EAX, 0
L1:
    CMP EAX, 2025
    JGE EMB
    MOV EBX, 0
L2:
    CMP EBX, 2025
    JGE L1
    INC EAX
    INC EBX
    JMP L2
EMB: NOOP
```

⑤.

5. Reduceți:
E1=(λh.(h λp.p) λz.(z z))
E2=(λx.(x (λh.(h λp.p) λz.(z z))) λq.(q q))

$E1 = (\lambda h.(h\ \lambda p.p)\ \lambda z.(z\ z))$

$E1 = \lambda z.(z\ z)\ \lambda p.p$

$E1 = \lambda p.p\ \lambda.p.p$

$E1 = \lambda p.p$

$E2 = (\lambda x.(x\ \underbrace{(\lambda h.(h\ \lambda p.p)\ \lambda z.(z\ z))}_{E1})\ \lambda q.(q\ q))$

$E2 = (\lambda x.(x\ \lambda p.p)\ \lambda q.(q\ q))$

$E2 = (\lambda q.(q\ q)\ \lambda p.p)$

$E2 = \lambda p.p\ \lambda p.p$

$E2 = \lambda p.p$